

✓ cloudChronicles Lab #003

Resilience-as-a-Service MVP

Project Goal

Build a prototype Resilience-as-a-Service platform that takes basic information about a cloud application and generates a customized disaster recovery recommendation.



```
# Let's begin by printing your name to personalize the notebook
```

```
your_name = "Lucas"  
print(f"Welcome to the lab, {your_name}!")
```

```
Welcome to the lab, Lucas!
```

STAR Method Lab Prompt

Situation

A financial institution operates an online banking platform that allows customers to check account balances, transfer money, pay bills, and manage their accounts. Because customers depend on these services, the platform must remain highly available even if a cloud region, database, or application service experiences a failure.

Task

Build a **Resilience-as-a-Service MVP** that allows a bank to enter information about its online banking platform and receive a customized disaster recovery and resilience plan.

The MVP should recommend appropriate cloud services, backup strategies, failover methods, monitoring solutions, and a resilient architecture based on the bank's requirements.

Action

1. Collect information about the bank and its online banking application.
2. Analyze the application's criticality and availability requirements.
3. Use Python to process the user's inputs.
4. Recommend appropriate GCP services.
5. Determine a backup strategy.
6. Determine a failover strategy.
7. Generate a customized disaster recovery plan.
8. Create an architecture diagram.
9. Test the MVP using different banking scenarios.

Expected Result

A working **Resilience-as-a-Service MVP** that accepts banking application requirements and produces a customized disaster recovery plan containing:

- Primary region
- Backup region
- Recommended GCP services
- Backup strategy
- Failover strategy
- Monitoring strategy
- Recovery objectives
- Resilient architecture diagram



MVP Overview

BankShield — Resilience-as-a-Service

BankShield is a prototype resilience platform designed for online banking applications.

The MVP collects information about a bank's application and uses that information to generate a customized disaster recovery recommendation.

MVP Process

Bank Information → Risk Analysis → GCP Recommendations → DR Strategy → Final DR Plan

The MVP will provide:

- Disaster recovery recommendations
- GCP service recommendations
- Backup strategy
- Failover strategy
- Monitoring recommendations
- Recovery objectives
- High-availability architecture

✓ User Inputs

The bank will provide information about its online banking platform.

Required Information

- Bank name
- Number of customers
- Primary GCP region
- Application criticality
- Database technology

The MVP will use these inputs to generate a customized resilience strategy.

```
# Collect information about the online banking platform
```

```
bank_name = input("Enter the bank name: ")
```

```
customers = int(input("Enter the number of customers: "))
primary_region = input("Enter the primary GCP region: ")
criticality = input("Enter application criticality (Low/Medium/High): ")
database = input("Enter the database technology: ")

print("\nBank information successfully collected!")
```

```
Enter the bank name: Banking4all
Enter the number of customers: 20
Enter the primary GCP region: us-east1
Enter application criticality (Low/Medium/High): Medium
Enter the database technology: MySQL

Bank information successfully collected!
```

✓ Resilience Analysis

The MVP will analyze the application's criticality and determine an appropriate disaster recovery strategy.

Higher-criticality banking applications require stronger resilience, more frequent backups, and faster failover.

```
# Analyze application criticality

if criticality.lower() == "high":
    backup_frequency = "Every 1-6 hours"
    failover_strategy = "Automatic regional failover"
    architecture = "Multi-region high-availability architecture"
    recovery_priority = "Very High"

elif criticality.lower() == "medium":
    backup_frequency = "Every 12 hours"
    failover_strategy = "Automated or manual regional failover"
    architecture = "Regional redundancy"
    recovery_priority = "High"

else:
    backup_frequency = "Daily"
    failover_strategy = "Manual failover"
    architecture = "Standard backup architecture"
    recovery_priority = "Moderate"

print("Analysis complete!")
print(f"Backup Frequency: {backup_frequency}")
print(f"Failover Strategy: {failover_strategy}")
print(f"Architecture: {architecture}")
print(f"Recovery Priority: {recovery_priority}")
```

Analysis complete!
Backup Frequency: Every 12 hours
Failover Strategy: Automated or manual regional failover
Architecture: Regional redundancy
Recovery Priority: High

✓ GCP Service Recommendations

Based on the banking application's requirements, the MVP will recommend GCP services that can improve availability, monitoring, data protection, and disaster recovery.

Recommended Services

- **Cloud Run** — Application hosting
- **Cloud SQL** — Relational database
- **Cloud Storage** — Backup and file storage
- **Cloud Load Balancing** — Traffic distribution and failover
- **Cloud DNS** — DNS management
- **Cloud Monitoring** — Application and infrastructure monitoring
- **Pub/Sub** — Event-driven alerts and recovery workflows

```
# Recommended GCP services

gcp_services = [
    "Cloud Run",
    "Cloud SQL",
    "Cloud Storage",
    "Cloud Load Balancing",
    "Cloud DNS",
    "Cloud Monitoring",
    "Pub/Sub"
]

print("Recommended GCP Services:\n")

for service in gcp_services:
    print(f"✓ {service}")
```

Recommended GCP Services:

✓ Cloud Run
✓ Cloud SQL
✓ Cloud Storage
✓ Cloud Load Balancing
✓ Cloud DNS
✓ Cloud Monitoring
✓ Pub/Sub

Generated Disaster Recovery Plan

The MVP will now combine the user's information and the resilience analysis into a customized disaster recovery plan.

```
# Generate the customized disaster recovery plan

print("=" * 60)
print("          BANKSHIELD DISASTER RECOVERY PLAN")
print("=" * 60)

print(f"\nBank Name: {bank_name}")
print(f"Number of Customers: {customers:,}")
print(f"Primary GCP Region: {primary_region}")
print(f"Application Criticality: {criticality}")
print(f"Database Technology: {database}")

print("\n--- RESILIENCE STRATEGY ---")
print(f"Architecture: {architecture}")
print(f"Backup Frequency: {backup_frequency}")
print(f"Failover Strategy: {failover_strategy}")
print(f"Recovery Priority: {recovery_priority}")

print("\n--- GCP SERVICES ---")

for service in gcp_services:
    print(f"✓ {service}")

print("\n--- DISASTER RECOVERY OBJECTIVE ---")
print(
    "Maintain access to critical online banking services "
    "during infrastructure or regional failures."
)

print("\n" + "=" * 60)
print("DR PLAN GENERATION COMPLETE")
print("=" * 60)
```

```
=====
          BANKSHIELD DISASTER RECOVERY PLAN
=====

Bank Name: Banking4all
Number of Customers: 20
Primary GCP Region: us-east1
Application Criticality: Medium
Database Technology: MySQL
```

```

--- RESILIENCE STRATEGY ---
Architecture: Regional redundancy
Backup Frequency: Every 12 hours
Failover Strategy: Automated or manual regional failover
Recovery Priority: High

--- GCP SERVICES ---
✓ Cloud Run
✓ Cloud SQL
✓ Cloud Storage
✓ Cloud Load Balancing
✓ Cloud DNS
✓ Cloud Monitoring
✓ Pub/Sub

--- DISASTER RECOVERY OBJECTIVE ---
Maintain access to critical online banking services during infrastructure or reg

=====
DR PLAN GENERATION COMPLETE
=====

```



Resilient Banking Architecture

The architecture below demonstrates how an online banking platform could use multiple GCP services and a backup region to improve availability.

Architecture Flow

Customers → Cloud DNS → Cloud Load Balancing → Banking Application → Database/Storage → Monitoring

The backup region provides an additional environment that can support the application if the primary region becomes unavailable.

Architecture Diagram

[Insert your architecture diagram here]

✓ MVP Testing

The MVP will be tested with different banking scenarios to determine whether its recommendations change based on application requirements.

Test Scenario 1 — Small Bank

- Customers: 10,000
- Criticality: Medium

Expected Result: Regional redundancy and scheduled backups.

Test Scenario 2 — Regional Bank

- Customers: 500,000
- Criticality: High

Expected Result: Multi-region architecture and automated failover.

Test Scenario 3 — National Bank

- Customers: 10,000,000
- Criticality: High

Expected Result: Multi-region architecture, frequent backups, automated failover, and continuous monitoring.

```
# Test BankShield using multiple banking scenarios
```

```
test_cases = [  
    {  
        "name": "Small Bank",  
        "customers": 10000,  
        "criticality": "Medium"  
    },  
    {  
        "name": "Regional Bank",  
        "customers": 500000,  
        "criticality": "High"  
    },  
    {  
        "name": "National Bank",  
        "customers": 10000000,  
        "criticality": "High"  
    }  
]
```

```
for test in test_cases:
```

```
    print("\n" + "=" * 50)  
    print(test["name"])  
    print("=" * 50)
```

```
    print(f"Customers: {test['customers']:,}")  
    print(f"Criticality: {test['criticality']}")
```



```

if test["criticality"].lower() == "high":
    print("Recommendation: Multi-region architecture")
    print("Failover: Automatic")
    print("Backup: Every 1-6 hours")

else:
    print("Recommendation: Regional redundancy")
    print("Failover: Automated or manual")
    print("Backup: Every 12 hours")

```

```

=====
Small Bank
=====
Customers: 10,000
Criticality: Medium
Recommendation: Regional redundancy
Failover: Automated or manual
Backup: Every 12 hours

=====
Regional Bank
=====
Customers: 500,000
Criticality: High
Recommendation: Multi-region architecture
Failover: Automatic
Backup: Every 1-6 hours

=====
National Bank
=====
Customers: 10,000,000
Criticality: High
Recommendation: Multi-region architecture
Failover: Automatic
Backup: Every 1-6 hours

```



Your Assignment

BankShield — Resilience-as-a-Service

- **MVP Name:** BankShield — Resilience-as-a-Service
- **User Inputs:** Bank Name, Number of Customers, Primary GCP Region, Application Criticality, Database Technology
- **Output:** Customized Disaster Recovery Plan
- **Tech Stack:** Python + Google Colab + Google Cloud Platform

- **GCP Services:** Cloud Run, Cloud SQL, Cloud Storage, Cloud Load Balancing, Cloud DNS, Cloud Monitoring, Pub/Sub
- **Hosting:** Google Cloud Run
- **Architecture:** Multi-region high-availability banking architecture
- **Testing:** Small Bank, Regional Bank, and National Bank scenarios
- **Screenshots/Links:** Insert screenshots of the working MVP, generated DR plan, and architecture diagram



Final Results

The BankShield MVP successfully demonstrates how a Resilience-as-a-Service platform can collect information about an online banking application and generate a customized disaster recovery strategy.

Completed Deliverables

- User input system
- Resilience analysis
- GCP service recommendations
- Disaster recovery plan
- Backup strategy
- Failover strategy
- MVP testing
- Resilient architecture diagram

Future Improvements

A future version of BankShield could include:

- Gemini-powered recommendations
- Interactive dropdown menus
- Automatic PDF generation
- Firebase authentication
- Cloud Run deployment
- Automated architecture generation
- Real-time monitoring integration

